

EPIC VI

PROGRAMMING SCHOOL FOR SCIENTIFIC RESEARCH

3 - 7 AUGUST 2026

Physically Informed Neural Networks for solving PDEs

Sesión 2

Poisson 2D

PyTorch

Pablo Rodríguez López



Universidad
Rey Juan Carlos



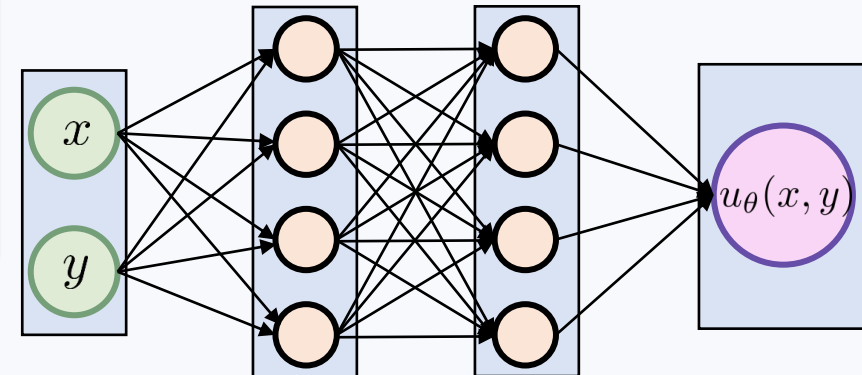
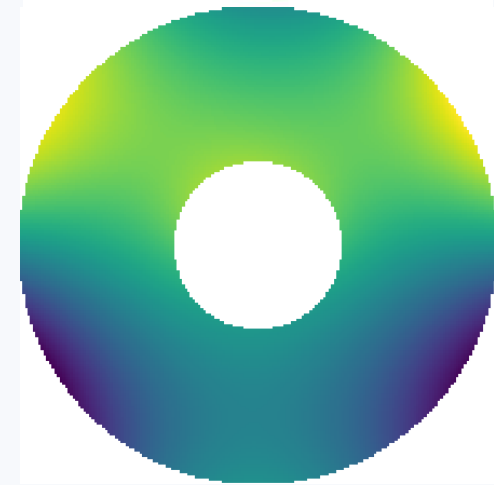
1 – Objetivos de la charla

- 1 · Physically Informed Neural Networks (PINN)
- 2 · Cómo derivar una Neural Network espacialmente
- 3 · Para qué se utiliza

2 – Ejemplo concreto

- 1 · Planteamiento del problema
- 2 · Definición del recinto
- 3 · Muestreo del recinto
- 4 · Función objetivo
- 5 · Extensiones

$$\Delta u = f$$

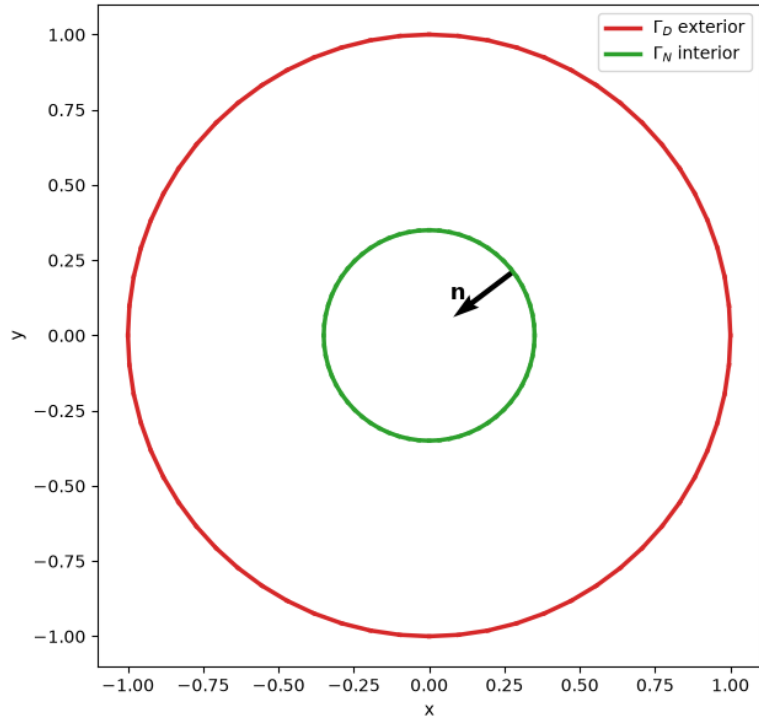


¿Puede una Neural Network aprender la solución de una PDE?

Forma Fuerte de la PDE planteada

Poisson en un Anillo, mismo problema, nueva representación

Dominio Ω , fronteras y normal interior



Γ_D exterior

Γ_N interior

$$\begin{aligned}\Delta u &= f & \forall \mathbf{r} \in \Omega \\ u &= g & \forall \mathbf{r} \in \Gamma_D \\ \partial_n u &= h & \forall \mathbf{r} \in \Gamma_N\end{aligned}$$

$$f(x, y) = x$$

$$g(x, y) = \sin(\pi y)$$

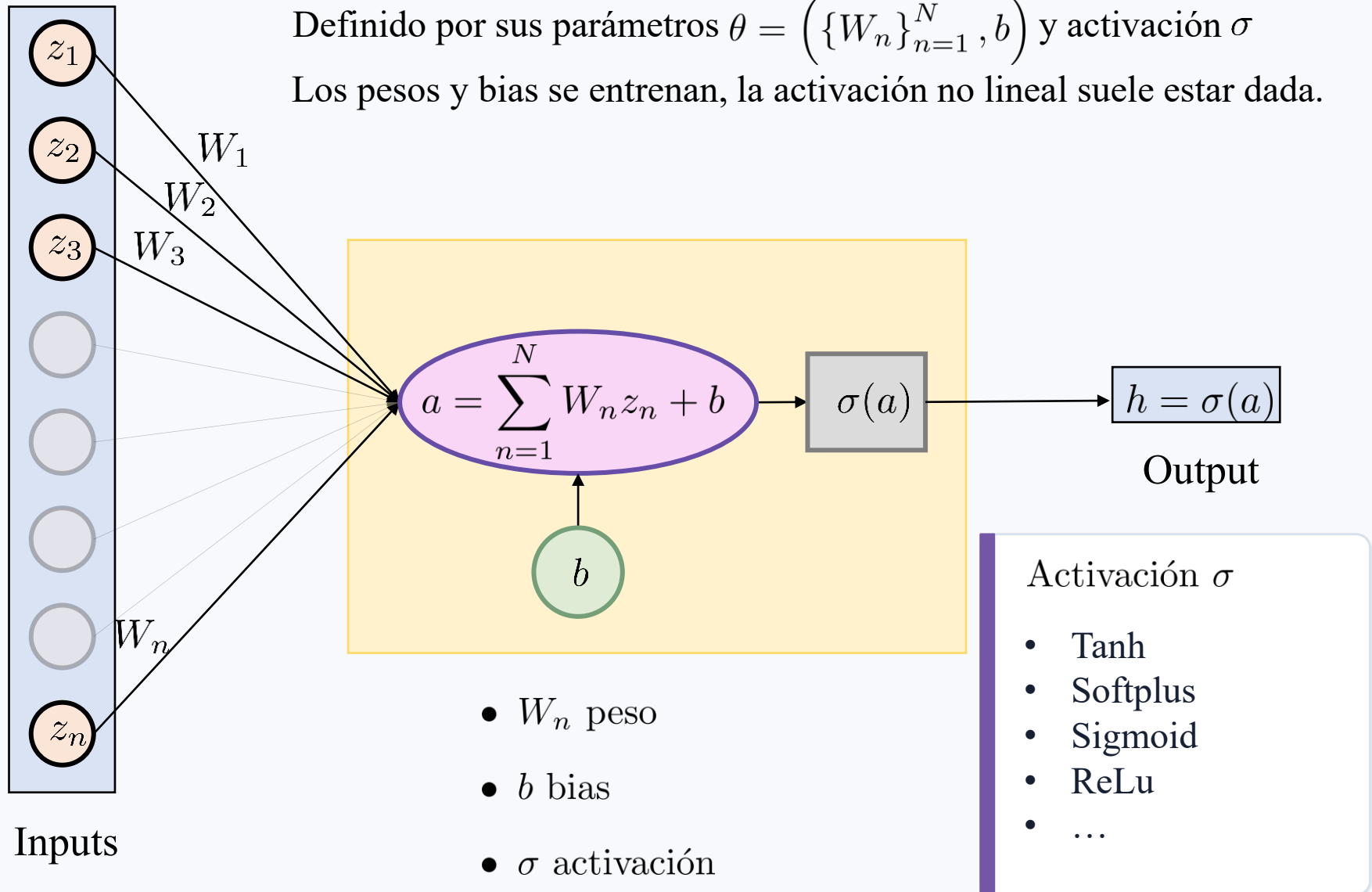
$$h(x, y) = \sqrt{x^2 + 1}$$

Cambia la familia aproximante, la EDP y los datos no cambian.

$$u_h(\mathbf{r}) = \sum_{e=1}^N u^e \varphi_e(\mathbf{r}) \longrightarrow u_\theta(\mathbf{r}) = \text{NN}_\theta(\mathbf{r})$$

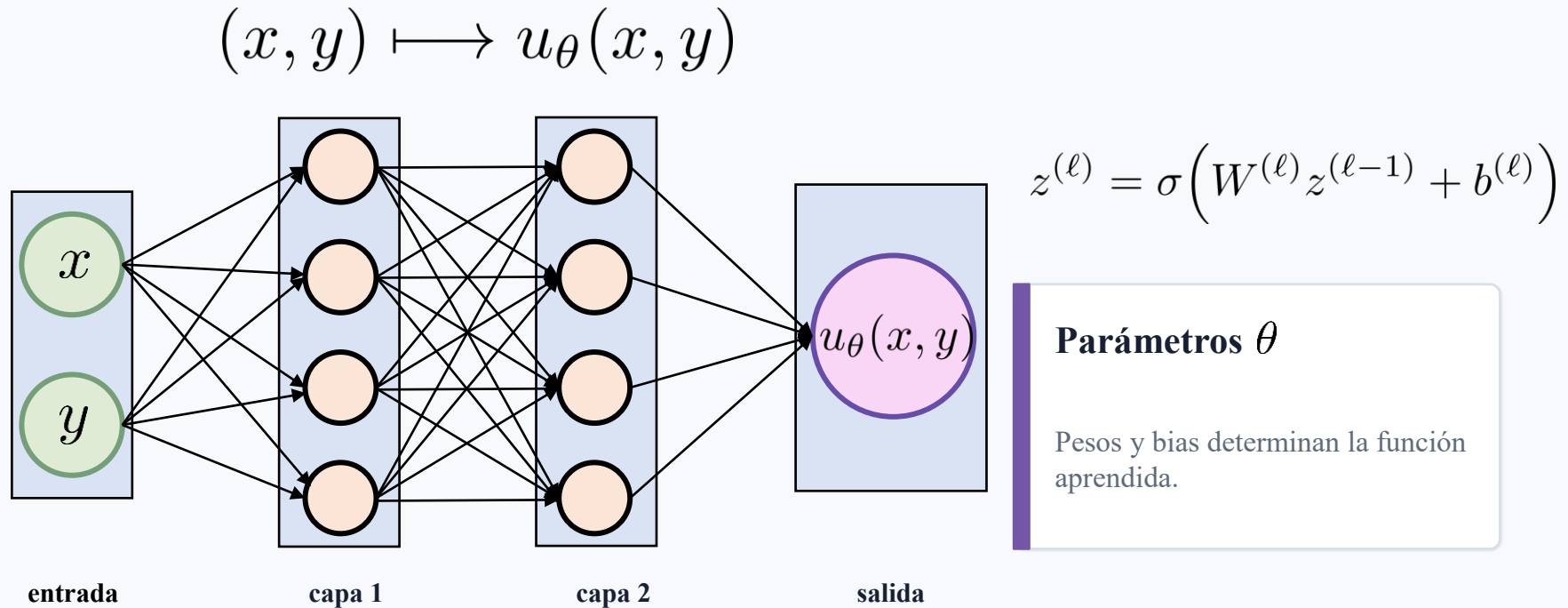
El perceptrón, la Neurona Artificial

Definido por sus parámetros $\theta = (\{W_n\}_{n=1}^N, b)$ y activación σ
Los pesos y bias se entrenan, la activación no lineal suele estar dada.



Red Neuronal

Una red neuronal es una función (de los datos de entrada) parametrizada (por los parámetros de sus neuronas) que da una salida.



En nuestro caso, la entrada serán coordenadas espaciales (x, y) , y la salida la función $u_{\theta}(x, y)$

Red Neuronal como Aproximante Universal

Leshno, M., Lin, V. Y., Pinkus, A., & Schocken, S. (1993).

“Multilayer Feedforward Networks with a Nonpolynomial Activation Function Can Approximate Any Function.”
Neural Networks, 6(6), 861–867. DOI: [https://doi.org/10.1016/S0893-6080\(0580131-5\)](https://doi.org/10.1016/S0893-6080(0580131-5))

No importa la función, **SIEMPRE** puede ser aproximada por una Red Neuronal,
Pero una solución de una PDE de orden n requiere de $\sigma(x)$ de orden n como mínimo.

Activación σ

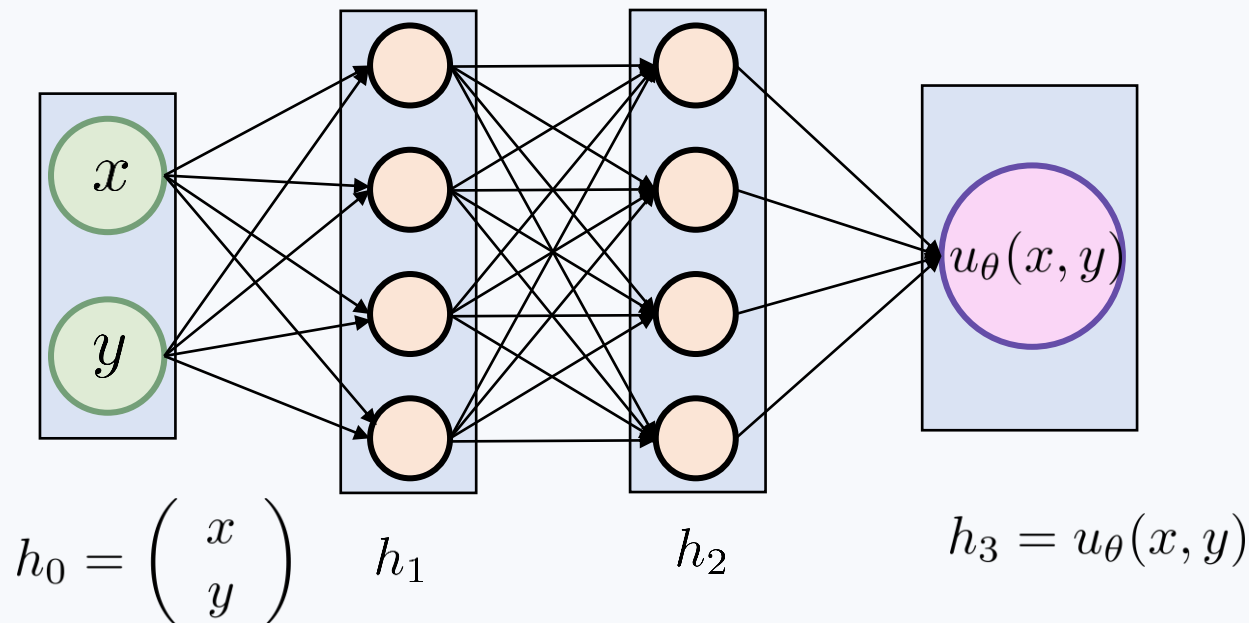
- Tanh ✓
- Softplus ✓
- Sigmoid ✓
- ReLU ✗

Diferenciación Automática

Derivar la red respecto a las entradas

Ejemplo: Red profunda de 2 capas

$$(x, y) \mapsto u_{\theta}(x, y)$$



$$u_{\theta}(x, y) = h_3(h_2(h_1(h_0))) = h_3 \circ h_2 \circ h_1 \circ h_0$$

$$h_{\ell} = \sigma(W_{\ell}h_{\ell-1} + b_{\ell}) = \sigma(a_{\ell})$$

$$\frac{\partial u_{\theta}(x, y)}{\partial x} = ?$$

PyTorch construye el grafo y aplica la regla de la cadena hasta obtener ∇u_{θ} y Δu_{θ}

Diferenciación Automática

Regla de la cadena a $u_\theta(x, y) = h_3(h_2(h_1(h_0))) = h_3 \circ h_2 \circ h_1 \circ h_0$

$$\frac{\partial u_\theta(x, y)}{\partial x} = \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial x}$$

Como $h_\ell = \sigma(W_\ell h_{\ell-1} + b_\ell) = \sigma(a_\ell) \Rightarrow \frac{\partial h_\ell}{\partial h_{\ell-1}} = \Sigma'_\ell W_\ell$ con $\Sigma'_\ell = \text{diag}(\sigma'(a_\ell))$

$$h_0 = \begin{pmatrix} x \\ y \end{pmatrix} \Rightarrow \frac{\partial h_0}{\partial x} = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

Realmente,

$$\frac{\partial u_\theta(x, y)}{\partial x} = \Sigma'_3 W_3 \Sigma'_2 W_2 \Sigma'_1 W_1 \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$\frac{\partial u_\theta(x, y)}{\partial y} = \Sigma'_3 W_3 \Sigma'_2 W_2 \Sigma'_1 W_1 \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Al ser la capa 3 la capa de salida, NO tiene activación, por lo que $\Sigma'_3 = 1$


Realmente, $\Sigma_3(x) = x$

Diferenciación Automática

Las segundas derivadas son más complejas

$$\frac{\partial u_\theta(x, y)}{\partial x} = \frac{\partial h_3}{\partial x} = \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial x}$$

$$\begin{aligned} \frac{\partial^2 u_\theta}{\partial x^2} = & \left(\frac{\partial^2 h_3}{\partial h_2 \partial x} \right) \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial x} \\ & + \frac{\partial h_3}{\partial h_2} \left(\frac{\partial^2 h_2}{\partial h_1 \partial x} \right) \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial x} \\ & + \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \left(\frac{\partial^2 h_1}{\partial h_0 \partial x} \right) \frac{\partial h_0}{\partial x} \\ & + \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \left(\frac{\partial^2 h_0}{\partial x^2} \right) \end{aligned}$$

Regla de la cadena

$$\frac{\partial u_\theta(x, y)}{\partial x} = \Sigma'_3 W_3 \Sigma'_2 W_2 \Sigma'_1 W_1 \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$\begin{aligned} \frac{\partial^2 u_\theta}{\partial x^2} = & \frac{\partial^2 h_3}{\partial h_2^2} \left(\frac{\partial^2 h_2}{\partial h_1^2} \right)^2 \left(\frac{\partial h_1}{\partial h_0} \right)^2 \left(\frac{\partial h_0}{\partial x} \right)^2 \\ & + \frac{\partial h_3}{\partial h_2} \left(\frac{\partial^2 h_2}{\partial h_1^2} \right) \left(\frac{\partial h_1}{\partial h_0} \right)^2 \left(\frac{\partial h_0}{\partial x} \right)^2 \\ & + \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \left(\frac{\partial^2 h_1}{\partial h_0^2} \right) \left(\frac{\partial h_0}{\partial x} \right)^2 \\ & + \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial^2 h_0}{\partial x^2} \end{aligned}$$

Las capas internas (1 y 2) verifican

$$\frac{\partial^2 h_\ell}{\partial h_{\ell-1}^2} = \frac{\partial}{\partial h_{\ell-1}} \Sigma'_\ell W_\ell = \Sigma''_\ell W_\ell^2 \text{ con } \Sigma''_\ell = \text{diag}(\sigma''(a_\ell))$$

Las capas 0 (input) y 3 (output) verifican

$$\frac{\partial^2 h_0}{\partial x^2} = 0 \quad \frac{\partial^2 h_3}{\partial h_2^2} = 0$$

$$\Sigma'_3 = 1$$

PyTorch construye el grafo y aplica la regla de la cadena para obtener ∇u_θ y Δu_θ .

Diferenciación Automática

Las segundas derivadas son más complejas

$$\frac{\partial u_\theta(x, y)}{\partial x} = \frac{\partial h_3}{\partial x} = \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial x}$$

$$\frac{\partial^2 u_\theta}{\partial x^2} = \left(\frac{\partial^2 h_3}{\partial h_2 \partial x} \right) \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial x} \quad \frac{\partial^2 u_\theta}{\partial x^2} = 0$$

$$+ \frac{\partial h_3}{\partial h_2} \left(\frac{\partial^2 h_2}{\partial h_1 \partial x} \right) \frac{\partial h_1}{\partial h_0} \frac{\partial h_0}{\partial x}$$

$$+ \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \left(\frac{\partial^2 h_1}{\partial h_0 \partial x} \right) \frac{\partial h_0}{\partial x}$$

$$+ \frac{\partial h_3}{\partial h_2} \frac{\partial h_2}{\partial h_1} \frac{\partial h_1}{\partial h_0} \left(\frac{\partial^2 h_0}{\partial x^2} \right)$$

Regla de la cadena

$$\frac{\partial u_\theta(x, y)}{\partial x} = \Sigma'_3 W_3 \Sigma'_2 W_2 \Sigma'_1 W_1 \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$+ \Sigma'_3 W_3 (\Sigma''_2 W_2^2) (\Sigma'_1 W_1)^2 \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$+ \Sigma'_3 W_3 \Sigma'_2 W_2 (\Sigma''_1 W_1^2) \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

$$+ 0$$

$$\frac{\partial^2 u_\theta}{\partial x^2} = \Sigma'_3 W_3 (\Sigma''_2 W_2^2) (\Sigma'_1 W_1)^2 \begin{pmatrix} 1 \\ 0 \end{pmatrix} + \Sigma'_3 W_3 \Sigma'_2 W_2 (\Sigma''_1 W_1^2) \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

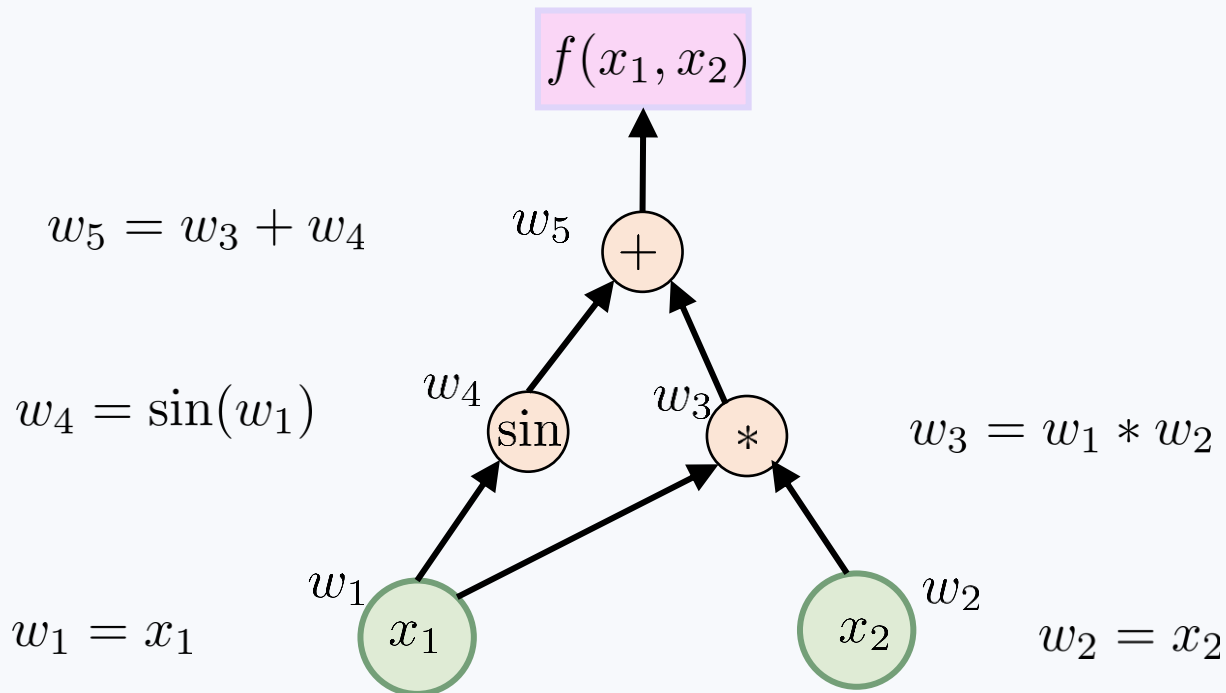
PyTorch construye el grafo y aplica la regla de la cadena hasta obtener ∇u_θ y Δu_θ .

Diferenciación Automática

Ejemplo: $f(x_1, x_2) = x_1 x_2 + \sin(x_1)$ $\vec{\nabla} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right) = ???$

Escribimos la función como composición de funciones y las ordenamos en un grafo

$$f(x_1, x_2) = w_5(w_3(w_1, w_2), w_4(w_1))$$



PyTorch construye el grafo y aplica la regla de la cadena hasta obtener ∇u_0 y Δu_0 .

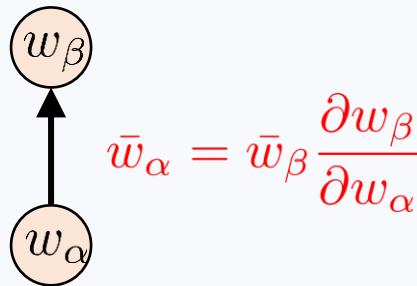
Diferenciación Automática

Ejemplo: $f(x_1, x_2) = x_1 x_2 + \sin(x_1)$ $\vec{\nabla} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right) = ???$

Escribimos la función como composición de funciones y las ordenamos en un grafo

$$f(x_1, x_2) = w_5(w_3(w_1, w_2), w_4(w_1))$$

Retropropagación: Recorremos el grafo en sentido contrario usando la siguiente regla



$\bar{w}_\alpha$ es el adjunto de w_α

Usamos el resultado de arriba para obtener el de la capa inferior, como en backpropagation

PyTorch construye el grafo y aplica la regla de la cadena hasta obtener ∇u_0 y Δu_0 .

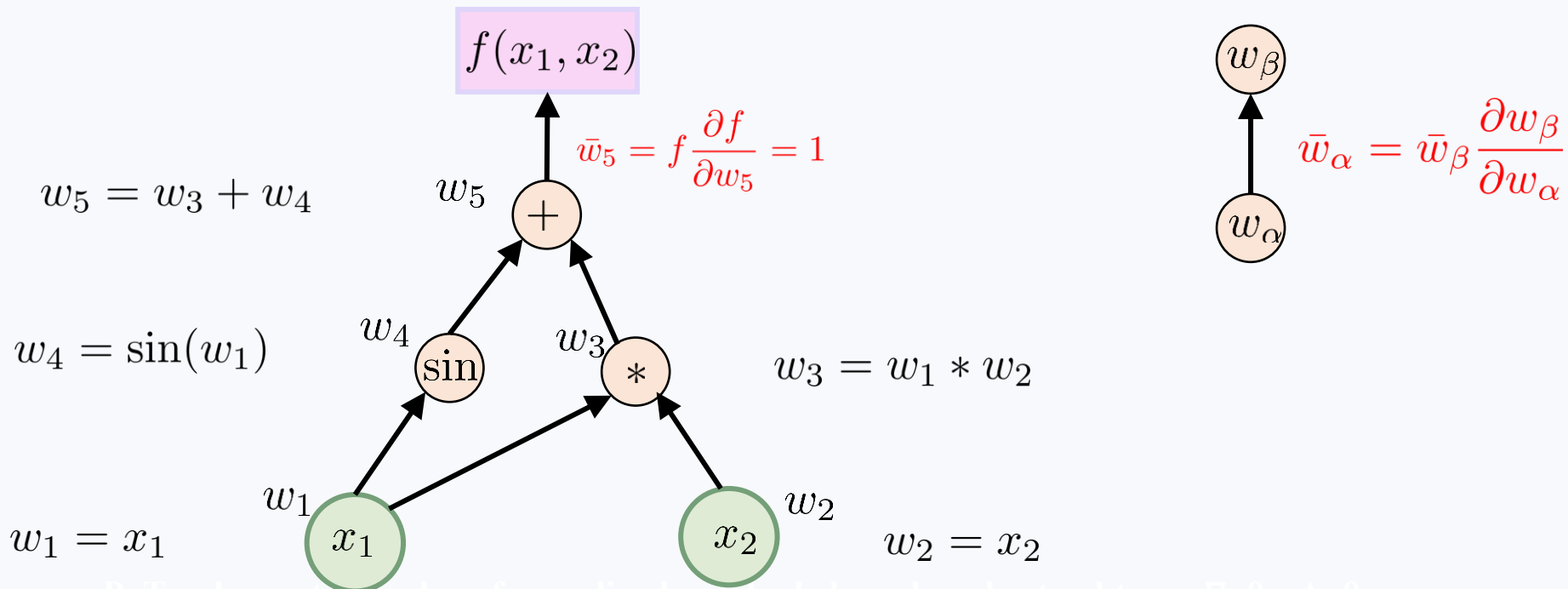
Diferenciación Automática

Ejemplo: $f(x_1, x_2) = x_1 x_2 + \sin(x_1)$ $\vec{\nabla} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right) = ???$

Escribimos la función como composición de funciones y las ordenamos en un grafo

$$f(x_1, x_2) = w_5(w_3(w_1, w_2), w_4(w_1))$$

Retropropagación: Recorremos el grafo en sentido contrario usando la siguiente regla



PyTorch construye el grafo y aplica la regla de la cadena hasta obtener ∇u_0 y Δu_0 .

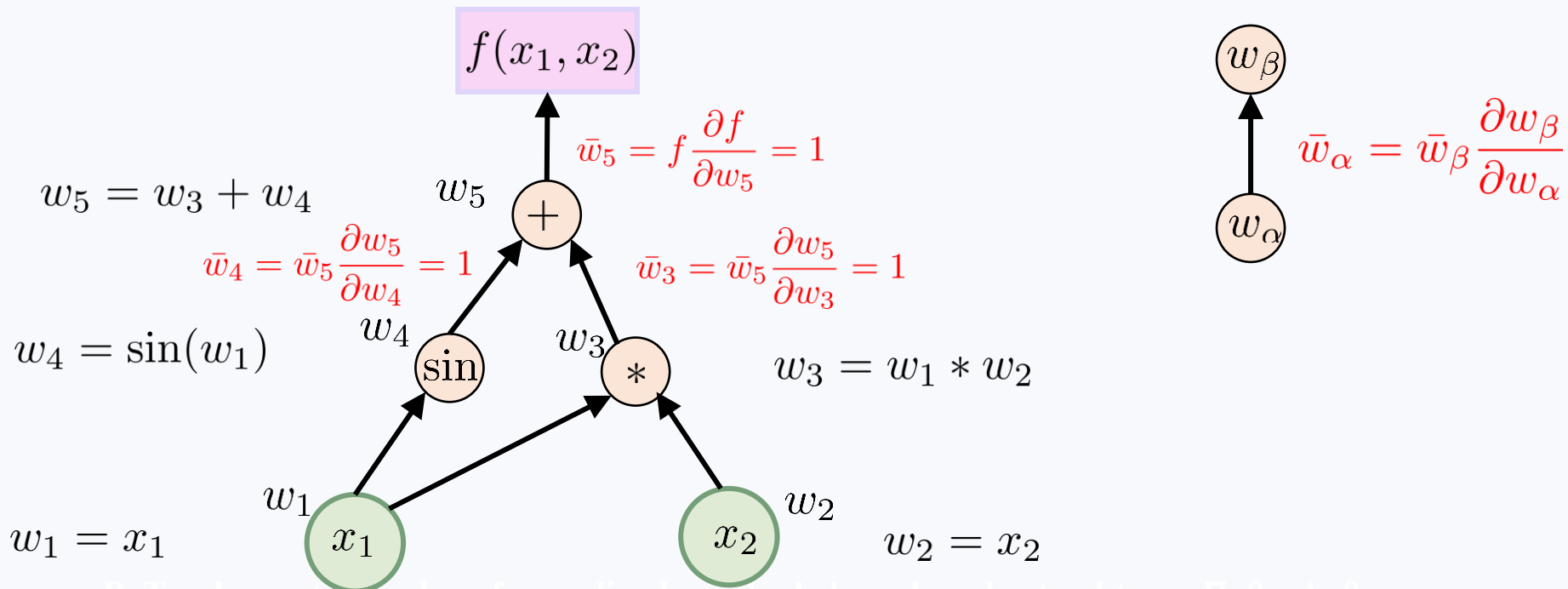
Diferenciación Automática

Ejemplo: $f(x_1, x_2) = x_1 x_2 + \sin(x_1)$ $\vec{\nabla} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right) = ???$

Escribimos la función como composición de funciones y las ordenamos en un grafo

$$f(x_1, x_2) = w_5(w_3(w_1, w_2), w_4(w_1))$$

Retropropagación: Recorremos el grafo en sentido contrario usando la siguiente regla



PyTorch construye el grafo y aplica la regla de la cadena hasta obtener ∇u_0 y Δu_0 .

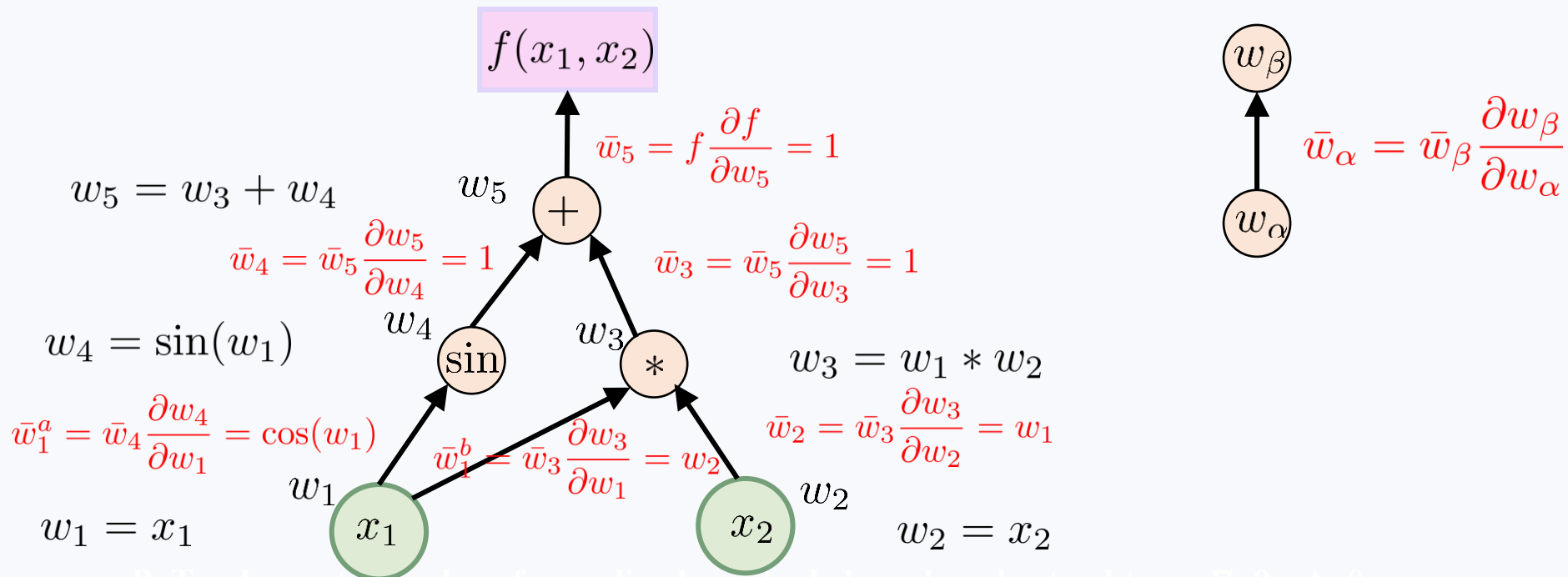
Diferenciación Automática

Ejemplo: $f(x_1, x_2) = x_1 x_2 + \sin(x_1)$ $\vec{\nabla} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right) = ???$

Escribimos la función como composición de funciones y las ordenamos en un grafo

$$f(x_1, x_2) = w_5(w_3(w_1, w_2), w_4(w_1))$$

Retropropagación: Recorremos el grafo en sentido contrario usando la siguiente regla



PyTorch construye el grafo y aplica la regla de la cadena hasta obtener ∇u_0 y Δu_0 .

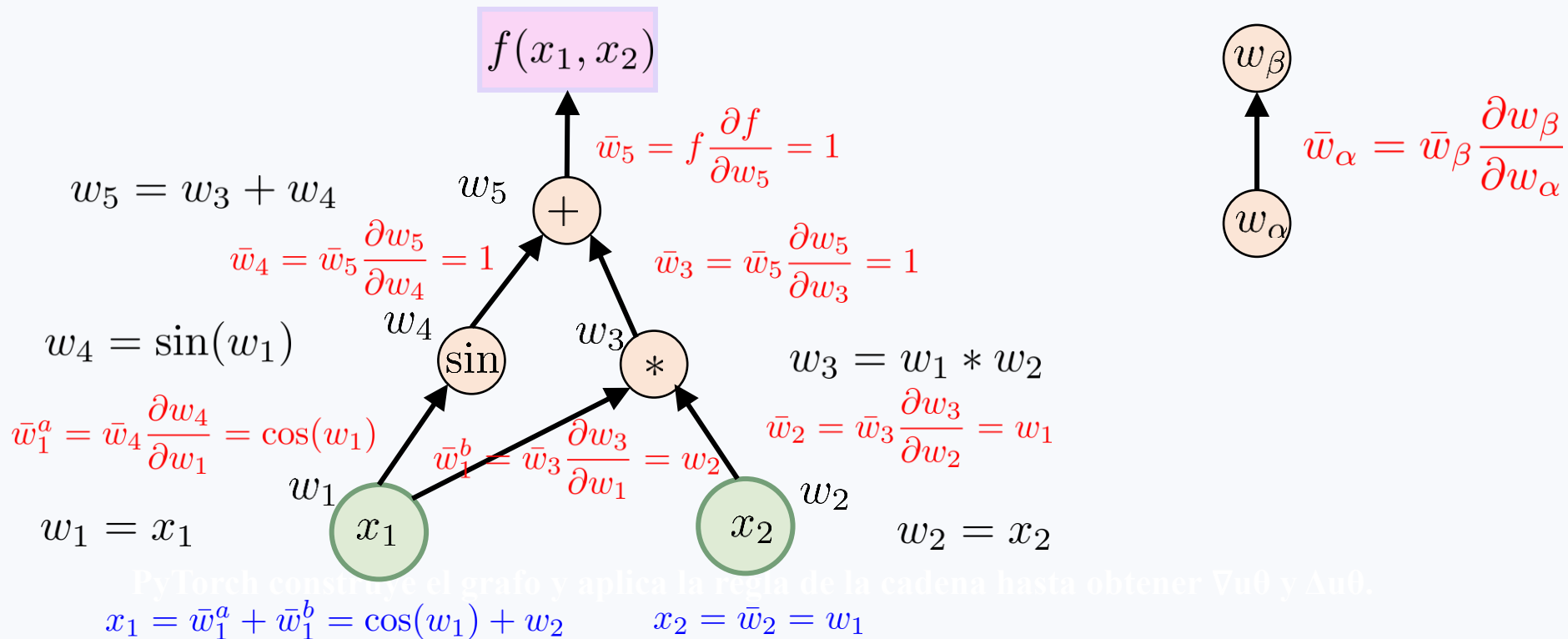
Diferenciación Automática

Ejemplo: $f(x_1, x_2) = x_1 x_2 + \sin(x_1)$ $\vec{\nabla} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right) = ???$

Escribimos la función como composición de funciones y las ordenamos en un grafo

$$f(x_1, x_2) = w_5(w_3(w_1, w_2), w_4(w_1))$$

Retropropagación: Recorremos el grafo en sentido contrario usando la siguiente regla



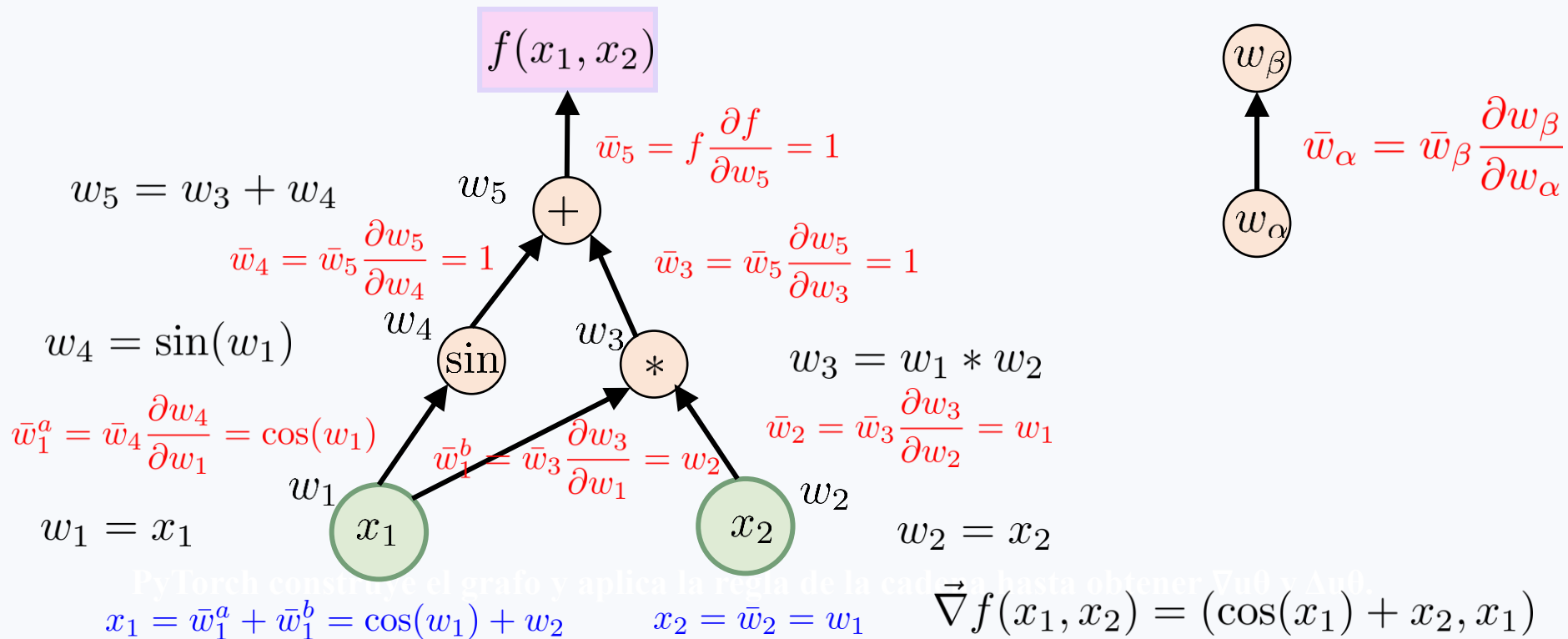
Diferenciación Automática

Ejemplo: $f(x_1, x_2) = x_1 x_2 + \sin(x_1)$ $\vec{\nabla} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right) = ???$

Escribimos la función como composición de funciones y las ordenamos en un grafo

$$f(x_1, x_2) = w_5(w_3(w_1, w_2), w_4(w_1))$$

Retropropagación: Recorremos el grafo en sentido contrario usando la siguiente regla



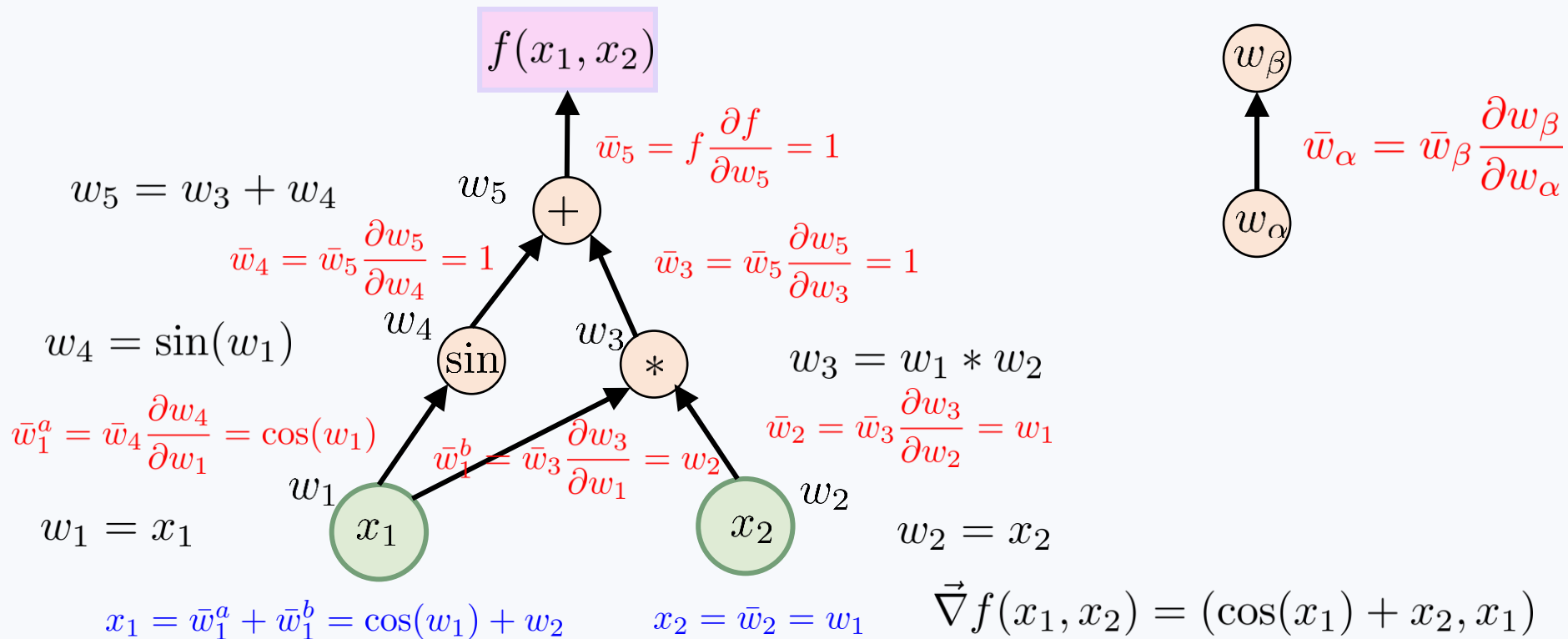
Diferenciación Automática

Ejemplo: $f(x_1, x_2) = x_1 x_2 + \sin(x_1)$ $\vec{\nabla} f(x_1, x_2) = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2} \right) = ???$

Escribimos la función como composición de funciones y las ordenamos en un grafo

$$f(x_1, x_2) = w_5(w_3(w_1, w_2), w_4(w_1))$$

Retropropagación: Recorremos el grafo en sentido contrario usando la siguiente regla



PyTorch construye el grafo y aplica la regla de la cadena hasta obtener ∇u_θ y Δu_θ .

Diferenciación Automática

```
def gradient_scalar_field(u, xy, create_graph=True):
    """Devuelve grad(u) para una salida escalar evaluada en xy."""
    return torch.autograd.grad(
        u,
        xy,
        grad_outputs=torch.ones_like(u),
        create_graph=create_graph,
        retain_graph=True,
    )[0]

def laplacian(model, xy, create_graph=True):
    """Calcula Delta u_theta punto a punto."""
    xy = xy.requires_grad_(True)
    u = model(xy)
    grad_u = gradient_scalar_field(u, xy, create_graph=True)

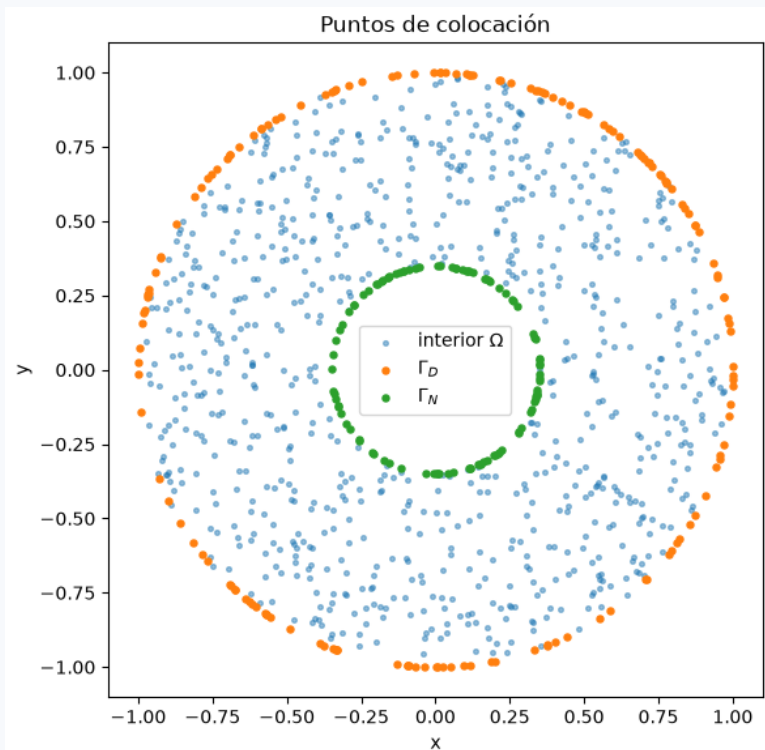
    u_x = grad_u[:, 0:1]
    u_y = grad_u[:, 1:2]
    grad_u_x = gradient_scalar_field(u_x, xy, create_graph=create_graph)
    grad_u_y = gradient_scalar_field(u_y, xy, create_graph=create_graph)

    u_xx = grad_u_x[:, 0:1]
    u_yy = grad_u_y[:, 1:2]
    return u, grad_u, u_xx + u_yy
```

Muestreo del dominio

Es importante elegir los puntos de training de forma uniforme y aleatoria.

No solo hay que muestrear el dominio, cada frontera requiere un muestreo aparte.



Interior Ω

Muestreo uniforme en área, no uniforme en radio.

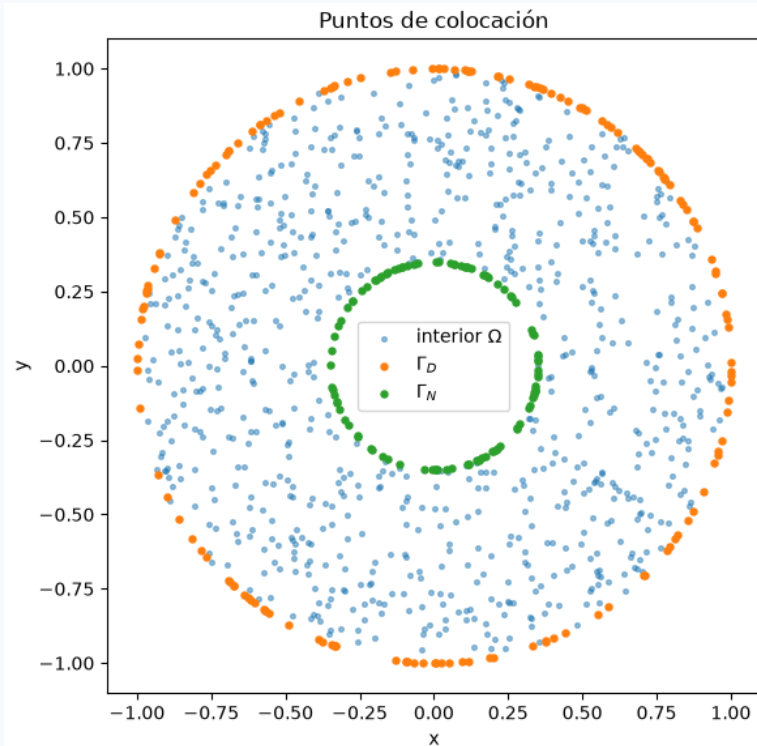
Frontera Γ_D

Puntos en la circunferencia exterior.

Frontera Γ_N

Puntos en la circunferencia interior.

Función objetivo a minimizar



PDE objetivo

$$\Delta u = f \quad \forall \mathbf{r} \in \Omega$$

$$\mathcal{L}_\Omega = \frac{1}{N_\Omega} \sum_{n=1}^{N_\Omega} |\Delta u_\theta(\mathbf{r}_n) - f(\mathbf{r}_n)|^2$$

La red se penaliza cuando Δu_θ difiere de f sobre Ω .

Condición de contorno de Neumann

$$\partial_n u = h \quad \forall \mathbf{r} \in \Gamma_N$$

$$\mathcal{L}_N = \frac{1}{N_N} \sum_{k=1}^{N_N} |\partial_n u_\theta(\mathbf{r}_k) - h(\mathbf{r}_k)|^2$$

La red se penaliza cuando $\partial_n u_\theta$ difiere de h sobre Γ_N .

Condición de contorno de Dirichlet

$$u = g \quad \forall \mathbf{r} \in \Gamma_D$$

$$\mathcal{L}_D = \frac{1}{N_D} \sum_{j=1}^{N_D} |u_\theta(\mathbf{r}_j) - g(\mathbf{r}_j)|^2$$

La red se penaliza cuando u_θ difiere de g sobre Γ_D .

Resolver una PDE como Optimización Multiobjetivo

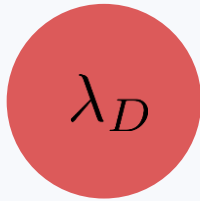
La función objetivo total es

$$\mathcal{L} = \lambda_{\Omega} \mathcal{L}_{\Omega} + \lambda_D \mathcal{L}_D + \lambda_N \mathcal{L}_N$$

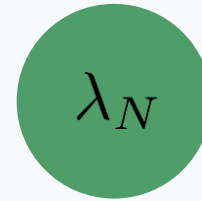
Tenemos 3 objetivos a la vez, cada uno con su peso específico



física interior



Dirichlet



Neumann

Los pesos cambian la solución del problema de optimización, no solo la escala de la gráfica, y dan una idea de la relevancia relativa que damos a cada objetivo concreto.

$$\mathcal{L}(\theta_0) = \min_{\theta}(\mathcal{L}(\theta)) \Rightarrow \frac{\delta \mathcal{L}(\theta_0)}{\delta \theta} = 0$$

Al ser un problema no lineal, necesitamos un algoritmo de minimización iterativo

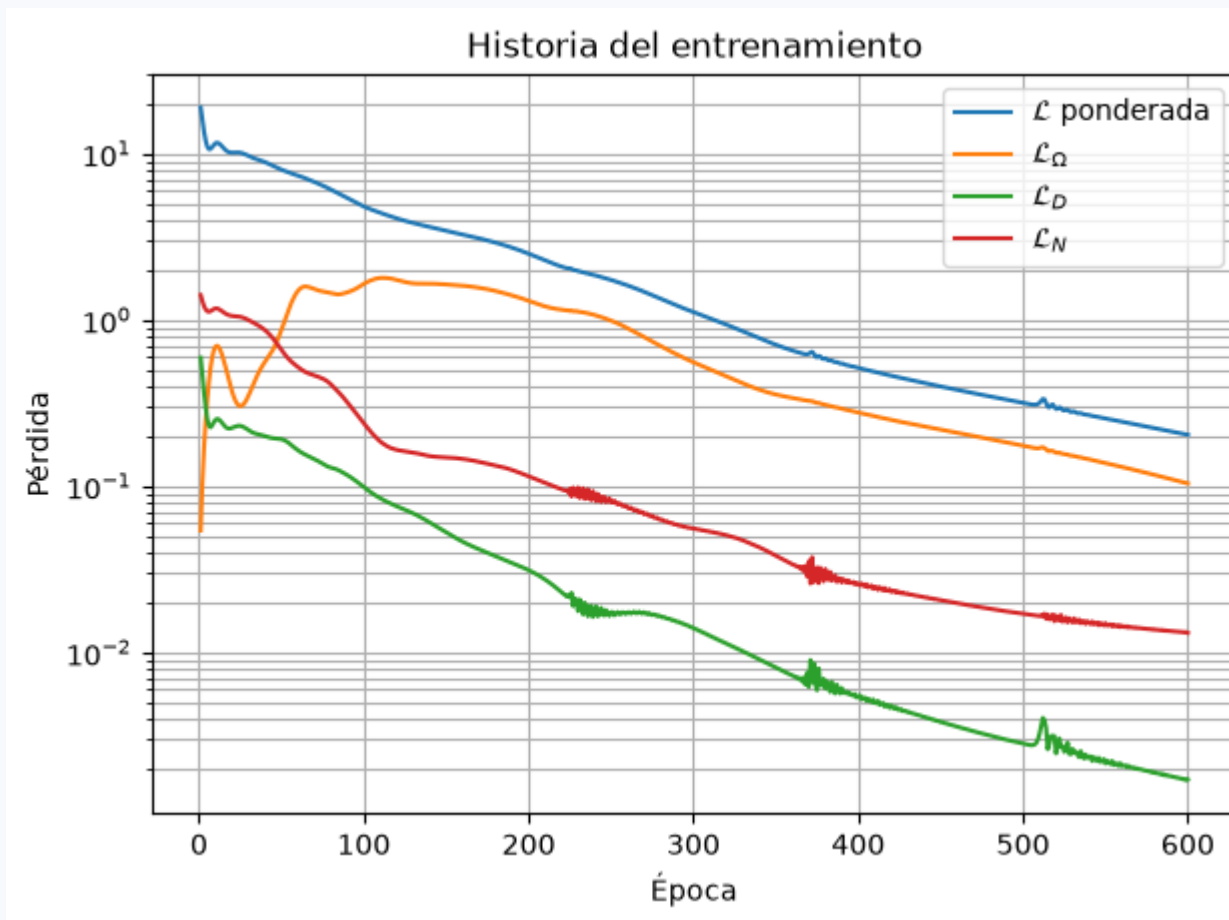
Entrenamiento con gradiente descendente o Adam, uso de backpropagation.

$$\theta_{n+1} = \theta_n - \alpha \frac{\delta \mathcal{L}(\theta_n)}{\delta \theta}$$

En FEM ya estamos en el mínimo, aquí tenemos que aproximarnos con mucho trabajo

La pérdida puede descender sin que la solución sea científicamente fiable.

Ejemplo de entrenamiento



Total

Interior

Neumann

Dirichlet

$$\lambda_\Omega = 1$$

$$\lambda_N = 5$$

$$\lambda_D = 20$$

Test de la red

Para validar la red, empleamos un conjunto independiente de puntos de test.

Residual interior RMS

¿satisface la EDP?

Dirichlet RMS / máximo

¿respeta g ?

Neumann RMS / máximo

¿respeta h ?

Mapa espacial

¿dónde falla?

Usar los mismos puntos de entrenamiento es incorrecto en un test.

Comparación entre Fem y PINN

$$\Delta u = f$$

FEM

PINN

Representación

Base local

Red Neuronal global

Problema numérico

Sistema lineal

Optimización no convexa

Dirichlet

Penalización dura

Penalización blanda

Función Objetivo

Forma débil

Función de pérdidas

Reproducibilidad

alta

depende de semilla

Comparar requiere usar el mismo problema, puntos comunes y métricas independientes.

Variational PINN

Formulación débil de la PDE en vez de la fuerte (como en FEM)

$$\mathcal{L}[u, v] = \oint_{\partial\Omega} ds v \partial_n u - \int_{\Omega} d\mathbf{r} \nabla v \cdot \nabla u - \int_{\Omega} d\mathbf{r} v f$$

La función objetivo optimiza una suma sobre las funciones prueba v_n

$$\mathcal{L}_{VPINN} = \frac{1}{N} \sum_{n=1}^N |\mathcal{L}[u_{\theta}, v_n]|^2$$

Ventaja

Formulación débil en vez de fuerte.

Necesita primeras derivadas, no segundas.

Concepto

Muy cercano a FEM, pero con Libertad de elección de las funciones de prueba

Dirichlet

Sigue requiriendo penalización o construcción exacta.

Deep Ritz

Nuestra función objetivo a minimizer es directamente una energía (o una acción)

$$J[u] = \int_{\Omega} d\mathbf{r} \left(\frac{|\nabla u|^2}{2} + uf \right) - \oint_{\Gamma_N} ds hu$$

Ventaja

Necesita primeras derivadas, no segundas.

Coste

Las integrales se aproximan mediante muestreo.

Dirichlet

Sigue requiriendo penalización o construcción exacta.

PINN Fuerte, VPINN y Deep Ritz son formulaciones diferentes del mismo problema físico.

Resolución de PDEs guiadas por datos

¿Podemos usar datos experimentales en nuestro modelo?

Sí, vuelta a Machine Learning clásico

Podemos ayudarnos de datos experimentales para mejorar la solución numérica del modelo.

Problemas inversos: aprender también parámetros físicos

$$\kappa \Delta u = f \quad \Rightarrow \quad u_\theta, \kappa \leftarrow \min (\mathcal{L}_{\text{Física}} + \mathcal{L}_{\text{datos}})$$

$$\mathcal{L}_{\text{Física}} = \frac{1}{N_\Omega} \sum_{n=1}^{N_\Omega} |\kappa \Delta u_\theta(\mathbf{r}_n) - f(\mathbf{r}_n)|^2$$

$$\mathcal{L}_{\text{datos}} = \frac{1}{N_d} \sum_{d=1}^{N_d} \|u_\theta(\mathbf{r}_d) - u_d^{\text{exp}}\|^2$$

Observaciones

valores experimentales de u

Física

residual de la EDP

Parámetro

α desconocido y positivo

Ejemplo obvio de uso

Predicción meteorológica

Tres ideas para llevarse

1

Una PINN representa u_θ y deriva esa función mediante autograd.

2

La pérdida combina física, Dirichlet y Neumann con pesos explícitos.

3

Validar exige puntos nuevos, varias semillas y una referencia numérica.

4

Permite mejorar un modelo con las observaciones experimentales

¿Preguntas?